```
work(dev)> exit
```

Panic over. We can now log back in to the application. But how can we stop this from happening again? We have several ways. For example, we could write code that prevents you from deleting your own user. That doesn't quite work: in theory, A could delete B at just the same time that B deletes A. Let's try a different approach. We'll delete the user inside a database transaction. Transactions provide an all-or-nothing proposition, stating that each work unit performed in a database must either complete in its entirety or none of them will have any effect whatsoever. If no users are left after we've deleted the user, we'll roll the transaction back, restoring the user we just deleted.

To do this, we'll use an Active Record hook method. We've already seen one of these: the `validate` hook is called by Active Record to validate an object's state. It turns out that Active Record defines sixteen or so hook methods, each called at a particular point in an object's life cycle. We'll use the `after_destroy` hook, which is called after the SQL `delete` is executed. If a method by this name is publicly visible, it'll conveniently be called in the same transaction as the `delete`—so if it raises an exception, the transaction will be rolled back. The hook method looks like this:

rails80/depot_t/app/models/user.rb

```
    class User < ApplicationRecord
      validates :name, presence: true, uniqueness: true
      validates :email_address, presence: true, uniqueness: true
      has_secure_password
      has_many :sessions, dependent: :destroy

      normalizes :email_address, with: ->(e) { e.strip.downcase }

»     after_destroy :ensure_an_admin_remains
»
»     class Error < StandardError
»     end
»
»     private
```